# BFS

$dist(u,v)$ ist die kürzeste Länge eines Weges von $u$ nach $v$

$S_k = \{ v \in V \mid dist(s,v) = k \}$ ist die Menge aller Knoten, die genau Distanz $k$ vom Startknoten $s$ entfernt sind

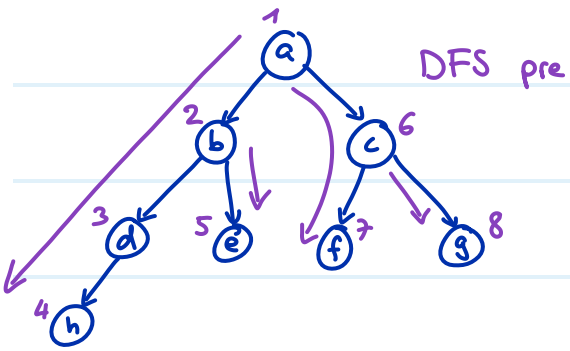
BFS berechnet die kürzesten Wege vom Startknoten $s$ zu allen anderen Knoten

**Algorithm 1** BFS(G, s) — Annahme G ungewichtet

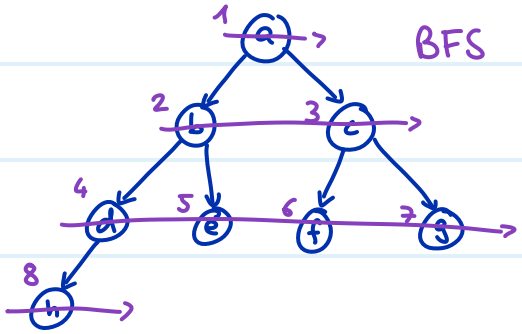
```
1: for each v ∈ V do
2:     visited[v] = false                ▷ Initialize visited status
3:     dist[v] ← ∞                       ▷ Initialize distance
4: Q ← ∅                                 ▷ Initialize empty queue Q
5: visited[s] ← true                     ▷ Mark start node as discovered
6: dist[s] ← 0                           ▷ Distance to start node is 0
7: ENQUEUE(s, Q)                         ▷ Add s to queue Q
8:
9: while Q ≠ ∅ do
10:    u ← DEQUEUE(Q)                    ▷ Current node
11:    for each (u, v) ∈ E do
12:        if visited[v] == false then   ▷ If v not yet discovered
13:            visited[v] ← true         ▷ Mark v as discovered
14:            dist[v] ← dist[u] + 1     ▷ BFS distance update
15:            ENQUEUE(v, Q)             ▷ Add v to queue Q
```

Initialisierungs-Teil

Laufzeit $O(n+m)$ mit Adjazenzliste



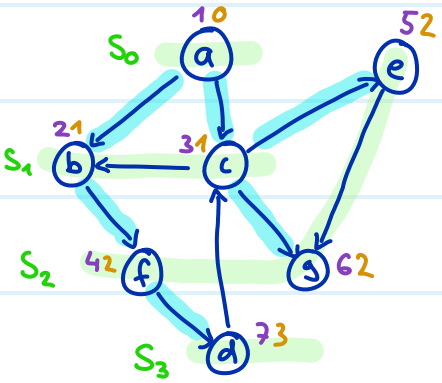

DFS pre




BFS enter

Achtung ob mit 0 oder 1 beginnen, und ob ihr nur enter berechnet oder auch leave

BFS mit Startknoten a



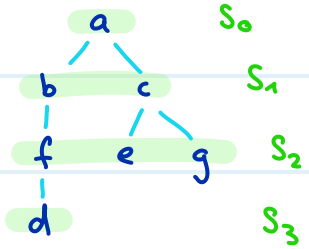

dist[]

enter[]

Level Sets

Shortest Path tree edges

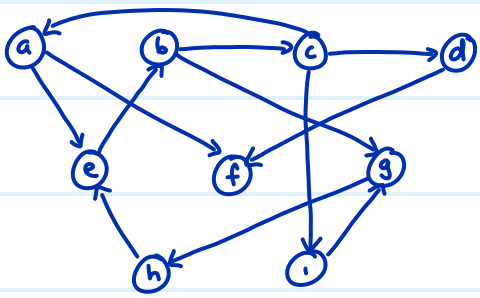
Shortest Path tree




$S_4 = \emptyset$

enter order = leave order

BFS mit Startknoten a

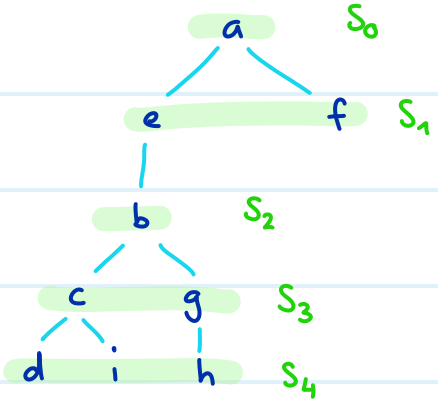


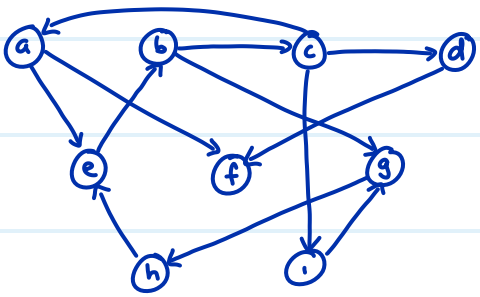

dist[]

enter[]

Level Sets

Shortest Path tree edges




Zum Vergleich: Tiefensuchbaum

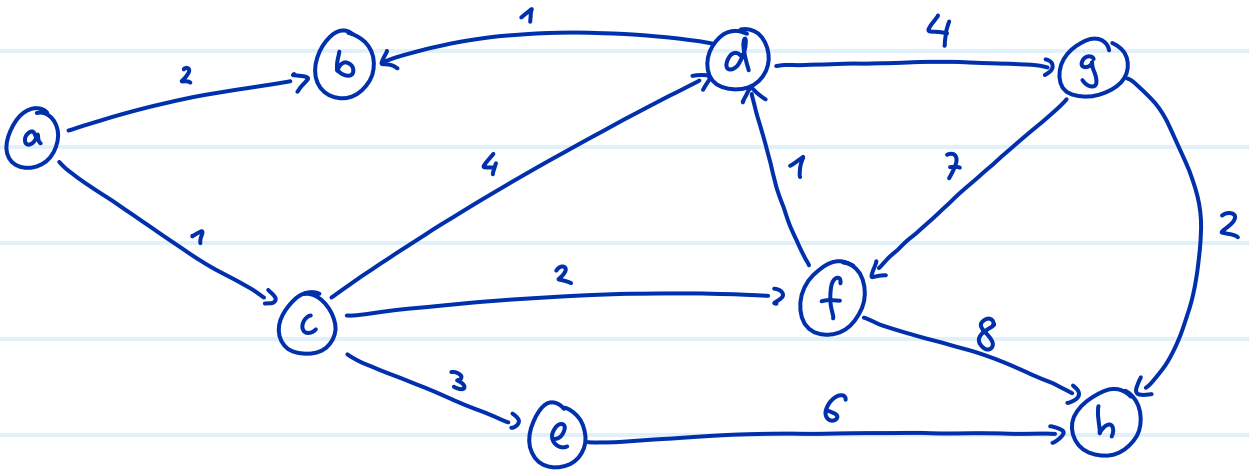
Schritt: 1) wir wählen von allen Kanten $(u,v) \in (S \times V \setminus S) \cap E$, das heisst $u \in S$ und $v \notin S$, diejenige aus, sodass $d[u] + c(u,v)$ minimal ist.

2) aktualisiere $d[v] = d[u] + c(u,v)$

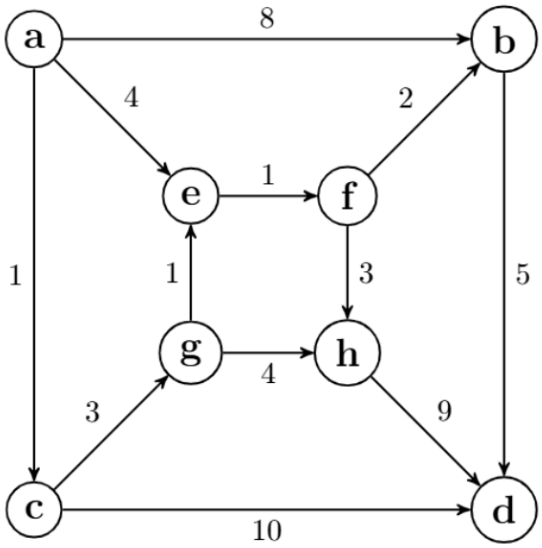
3) füge $v$ zu $S$ hinzu

shortest path tree

$d[v]$




/ 2 P e) *Shortest Path Tree:* Consider the following graph:



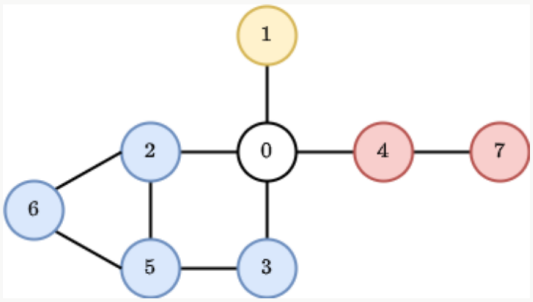

i) Find a shortest-path tree rooted at vertex $a$ (e.g., the set of edges selected by Dijkstra's algorithm starting from vertex $a$). You can either highlight its edges in the picture above, or draw the shortest-path tree below. (If there is more than one shortest-path tree, it suffices to just find one of them.)

# Graph Sets

You are given an undirected connected graph with $n$ nodes numbered from $0$ to $n-1$ and $m$ edges between them.

The nodes of the graph are partitioned into sets in the following way. Node $0$ forms a set of its own. Suppose that node $0$ is removed from the graph. Then, the nodes in each connected subgraph of the resulting graph forms another set.

An example is shown below, in which the nodes are colored according to the set they are in. Specifically, the sets in this example are $\{0\}$, $\{1\}$, $\{2,3,5,6\}$, and $\{4,7\}$.




Given such a graph, you have to answer queries of the following type:

1. **hasCycle()**: Return $1$ if the graph has a cycle, and $0$ otherwise.
2. **hasCycleWithoutNodeZero()**: Suppose that node $0$ is removed from the graph. Return $1$ if the resulting graph has a cycle, and $0$ otherwise.
3. **isSameSet($x$, $y$)**: Given two nodes $x$ and $y$, return $1$ if they are in the same set, and $0$ otherwise.
4. **getShortestPath($x$, $y$)**: Given two nodes $x$ and $y$ **that are in different sets**, return the shortest-path distance between them.

You have to implement the four methods described above. In addition, we provide an **intialize()** method which we guarantee to run before any of the queries. Feel free to use this method, for example, to initialize information that you want available in all of the queries (of course, the time consumed by **intialize()** counts toward the time limit of the problem).

For an easier implementation of the queries, **recall and make use of the fact** that the graph is connected.

**Grading** (24 points):

1. **hasCycle()** (4 points): This query will be run at most once. An $O(n+m)$-time implementation gets $4$ points.
2. **hasCycleWithoutNodeZero()** (4 points): This query will be run at most once. An $O(n+m)$-time implementation gets $4$ points.
3. **isSameSet($x$, $y$)** (8 points): If $q$ is the number of queries of this type, an $O(n+m+q)$-time implementation gets 8 points and an $O(q(n+m))$-time implementation gets 4 points.
4. **getShortestPath($x$, $y$)** (8 points): If $q$ is the number of queries of this type, an $O(n+m+q)$-time implementation gets 8 points and an $O(q(n+m))$-time implementation gets 4 points.